# Systém pro správu skladu elektronických součástek

## WMS – Elektronické součástky

Projektová dokumentace

**Autoři:** Tomáš Simandl
Tomáš Szetei
Lukáš Kvapil

**Datum:** 5. dubna 2026

Dokumentace systému pro skladovou evidenci elektronických součástek

# Obsah

**1 Datový model** **2**

1.1 Formulace problému . . . . . 2

1.2 Datový slovník . . . . . 2

1.3 Diagram tříd . . . . . 4

# Kapitola 1

# Datový model

## 1.1 Formulace problému

Cílem tohoto projektu je teoreticky navrhnout informační systém pro správu skladových součástek. Systém má umožňovat evidenci součástek, jejich kategorií, dodavatelů a skladových lokací. Dále má podporovat sledování pohybů součástek ve skladu, jako je naskladnění, vyskladnění nebo přesun mezi jednotlivými lokacemi.

Součástí systému je také možnost přiřazovat součástky k projektům a evidovat jejich množství potřebné pro daný projekt. Návrh systému je znázorněn pomocí UML class diagramu, který popisuje jednotlivé třídy, jejich atributy a vztahy mezi nimi.

## 1.2 Datový slovník

### Entity tříd

- **Soucastka**: Skladová položka evidovaná v systému WMS s aktuálním množstvím na skladě.
- **Projekt**: Projekt, ke kterému jsou přiřazeny součástky pro sledování spotřeby.
- **SkladovyPohyb**: Abstraktní třída reprezentující pohyb součástek – příjem, výdej nebo přesun.
- **PrijemPohyb**: Konkrétní pohyb typu příjem – naskladnění součástek od dodavatele (dědí ze **SkladovyPohyb**).
- **VydejPohyb**: Konkrétní pohyb typu výdej – vyskladnění součástek ze skladu (dědí ze **SkladovyPohyb**).
- **PresunutiPohyb**: Konkrétní pohyb typu přesun – přemístnění součástek mezi lokáními ve skladu (dědí ze **SkladovyPohyb**).

- **Sklad**: Fyzický sklad rozdělený na lokace, ve kterých jsou uloženy součástky.
- **Lokace**: Konkrétní místo v skladu identifikované označením (např. A-01-03).
- **Uzivatel**: Osoba spravující součástky, pohyby, projekty a sklady v systému.

## Atributy

Tabulka 1.1: Přehled atributů tříd

| **Entita** | **Atributy** |
|---|---|
| Soucastka | id, nazev, mnozstvi |
| Projekt | id, nazev |
| SkladovyPohyb | id, datum, stav |
| PrijemPohyb | dodavatel (dědí z SkladovyPohyb) |
| VydejPohyb | duvod (dědí z SkladovyPohyb) |
| PresunutiPohyb | zdrojLokace, cilLokace (dědí z SkladovyPohyb) |
| Sklad | id, nazev |
| Lokace | id, oznaceni |
| Uzivatel | id, jmeno, email |

## 1.3 Diagram tříd

Diagram zachycuje strukturu systému WMS a vztahy mezi jeho hlavními třídami. Součástky jsou přiřazovány k projektům a fyzicky umístěny v lokacích skladu. Třída `Uzivatel` reprezentuje uživatelskou roli v systému – může vytvářet skladové pohyby, spravovat součástky, přiřazovat součástky k projektům a zobrazovat stav skladu.

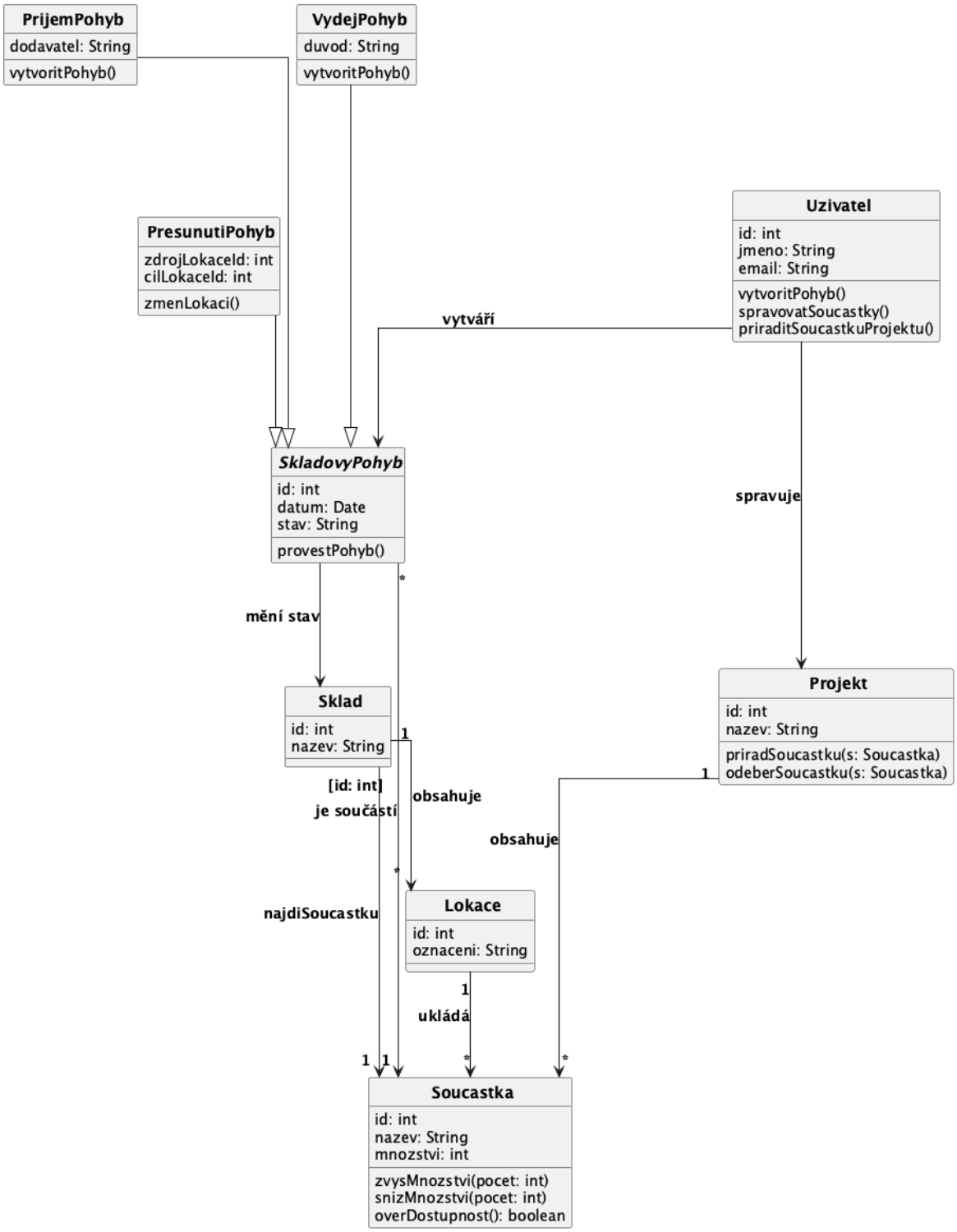
Diagram využívá **generalizaci**: abstraktní třída `SkladovyPohyb` je základem pro tři konkrétní podtřídy `PrijemPohyb` (naskladnění), `VydejPohyb` (vyskladnění) a `PresunutiPohyb` (přesun mezi lokacemi), které dědí společné atributy a přidávají vlastní specifické atributy.

Třída `SkladovyPohyb` zaznamenává každou změnu stavu zásob v systému. Konkrétně:

- `PrijemPohyb` – příjem součástek na sklad od dodavatele; zvyšuje množství dané součástky.
- `VydejPohyb` – výdej součástek ze skladu na projekt; snižuje množství dané součástky.
- `PresunutiPohyb` – přemístnění součástek mezi lokácemi ve skladu; mění přiřazenou lokaci součástky.

Každý pohyb se váže na konkrétní součástku (vazba `je součástí`) a mění stav skladu (vazba `mění stav`). Pohyb nemění fyzickou lokaci součástky – ta je pevně dána vazbou `Lokace` → `Soucastka`.

Vazba mezi `Sklad` a `Soucastka` je **kvalifikovaná** – sklad přistupuje ke konkrétní součástce prostřednictvím kvalifikátoru `id: int`.



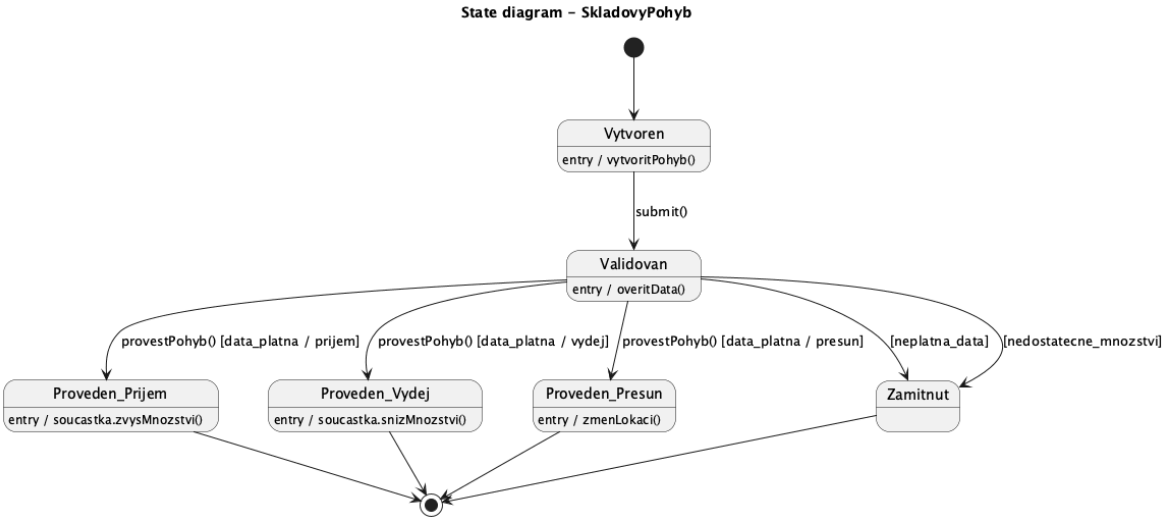

Obrázek 1.1: Class diagram WMS

## 1.4 Stavový diagram

### Stavový diagram třídy `SkladovyPohyb`

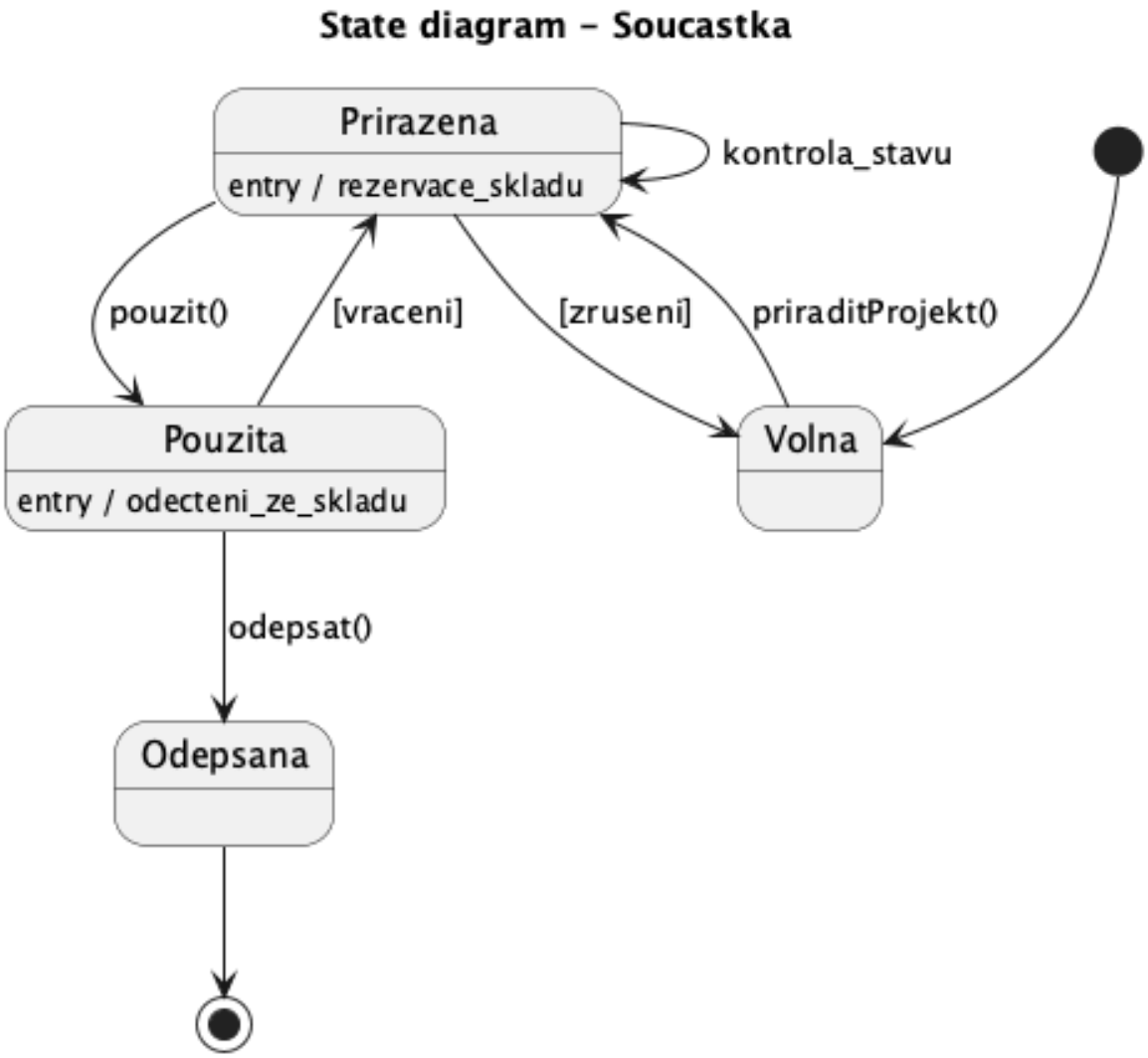
Třída `SkladovyPohyb` reprezentuje obecný skladový pohyb – tedy jakoukoli změnu stavu zásob součástek v systému. Každý pohyb prochází životním cyklem řízeným třemi metodami:

- `vytvoritPohyb()` – inicializuje nový pohyb: přiřadí ID, nastaví datum vzniku, typ pohybu a vazbu na součástku;
- `overitData()` – ověří platnost zadaných dat: existenci součástky, vyplnění povinných polí, dostatečné množství na skladě (příjem/výdej) nebo platnost lokací (přesun);
- `provestPohyb()` – fyzicky provede pohyb: u příjmu zavolá `soucastka.zvysMnozstvi()`, u výdeje `soucastka.snizMnozstvi()` a u přesunu `zmenLokaci()`.



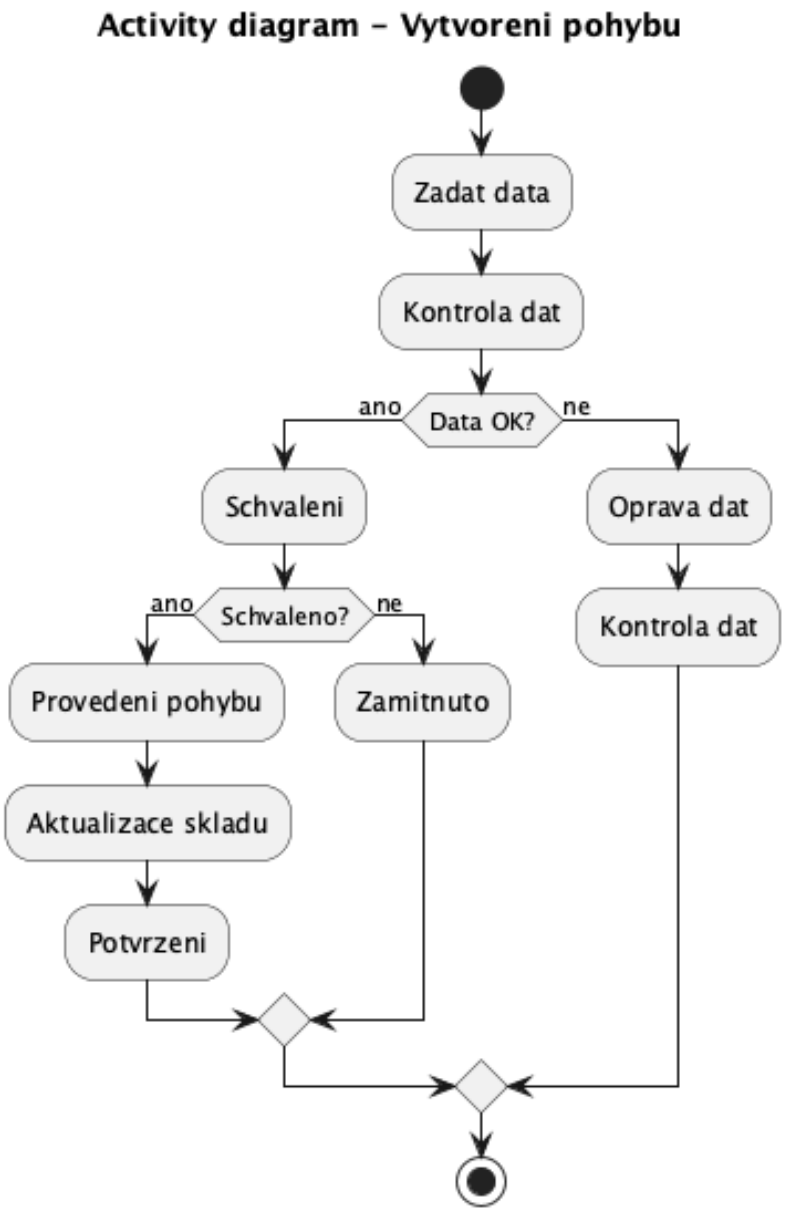

Obrázek 1.2: Stavový diagram třídy SkladovyPohyb

### Stavový diagram `Soucastka` (v projektu)



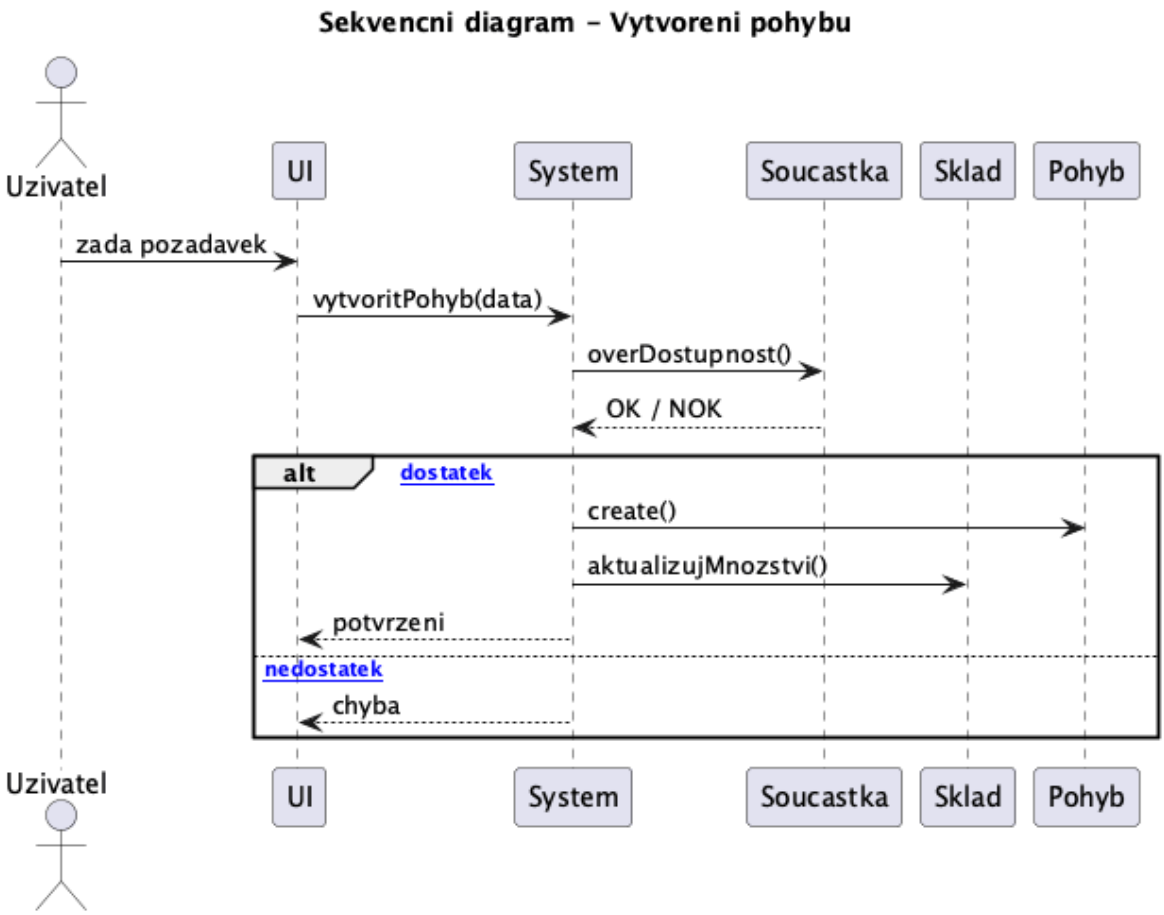

Obrázek 1.3: Stavový diagram třídy Soucastka (v projektu)

## 1.5 Activity diagram – Vytvoření pohybu



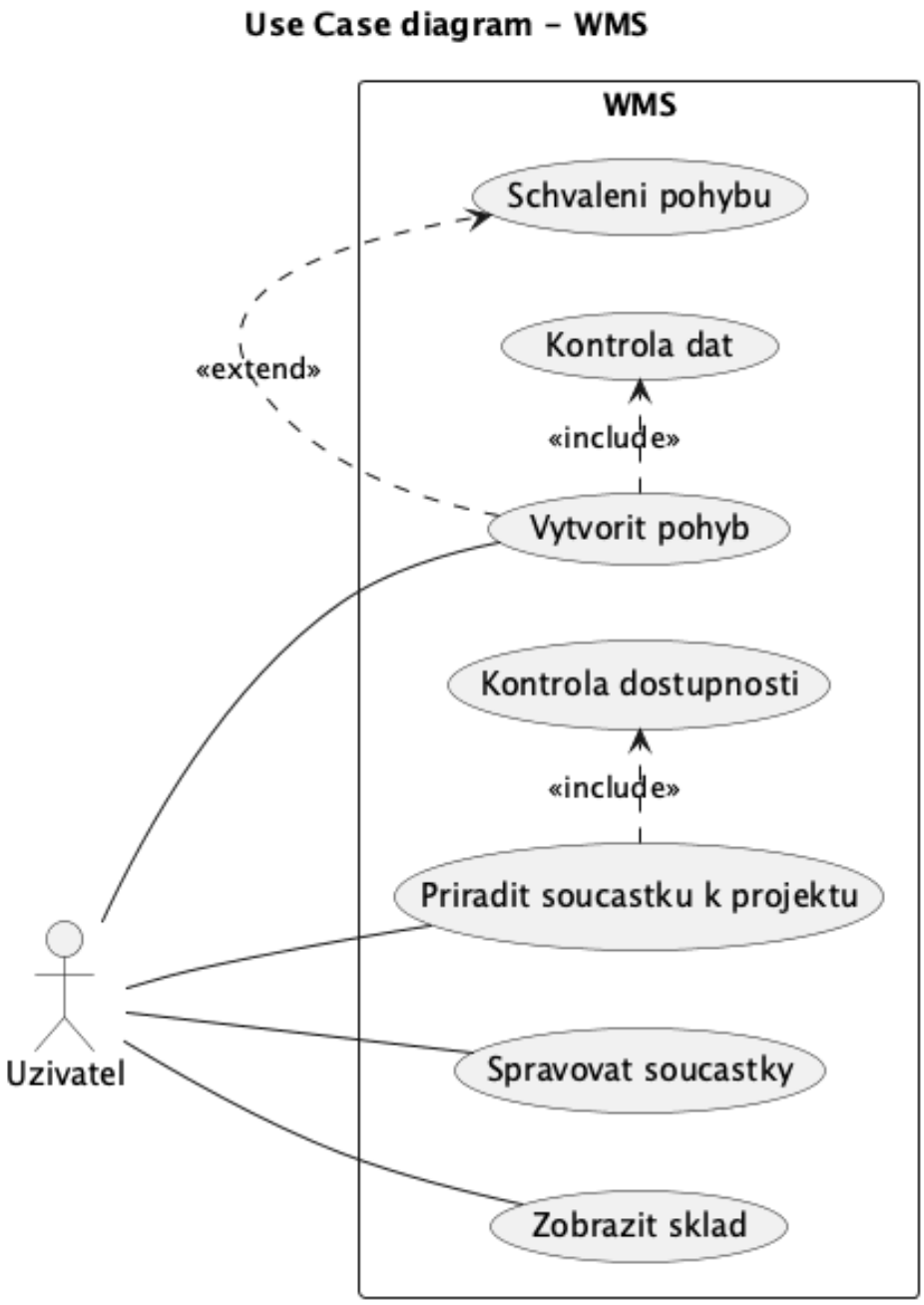

Obrázek 1.4: Activity diagram – Vytvoření pohybu

## 1.6 Sekvenční diagram – Vytvoření pohybu




Obrázek 1.5: Sekvenční diagram – Vytvoření pohybu

## 1.7 Use Case




Obrázek 1.6: Use Case Diagram – WMS

### Scénář: Vytvoření pohybu

1. Uživatel zvolí *Vytvořit pohyb*.
2. Zadá součástku a množství.
3. Systém provede kontrolu dat.
4. Pokud `[data_ok]` → pohyb je schválen.

5. Systém provede pohyb a aktualizuje stav skladu.
6. Zobrazí se potvrzení výsledku.

### Scénář: Přiřazení součástky k projektu

1. Uživatel vybere projekt.
2. Zvolí součástku a zadá množství.
3. Systém ověří dostupnost voláním `overDostupnost()`.
4. Pokud `[dostatek]` → součástka je přiřazena.
5. Systém aktualizuje množství na skladě.

Cílem tohoto projektu je teoreticky navrhnout informační systém pro správu skladových součástek. Systém má umožňovat evidenci součástek, jejich kategorií, dodavatelů a skladových lokací. Dále má podporovat sledování pohybů součástek ve skladu, jako je naskladnění, vyskladnění nebo přesun mezi jednotlivými lokacemi.

Součástí systému je také možnost přiřazovat součástky k projektům a evidovat jejich množství potřebné pro daný projekt. Návrh systému je znázorněn pomocí UML class diagramu, který popisuje jednotlivé třídy, jejich atributy a vztahy mezi nimi. Diagram využívá vybrané vlastnosti UML, jako jsou zobecnění, kardinality, agregace, kvalifikované vazby a atributy spojení, aby byl model systému co nejpřesnější.

## 1.8 Přehled entit

Systém pracuje s následujícími hlavními třídami:

**`Uzivatel`** Hlavní uživatel systému; vytváří pohyby a spravuje součástky.

**`Sklad`** Fyzický sklad identifikovaný názvem; sdružuje skladové lokace.

**`Lokace`** Konkrétní místo v skladu (přihrádka, regál) identifikované označením.

**`Soucastka`** Elektronická součástka evidovaná v systému; obsahuje atributy `id`, `nazev` a aktuální `mnozstvi` na skladě.

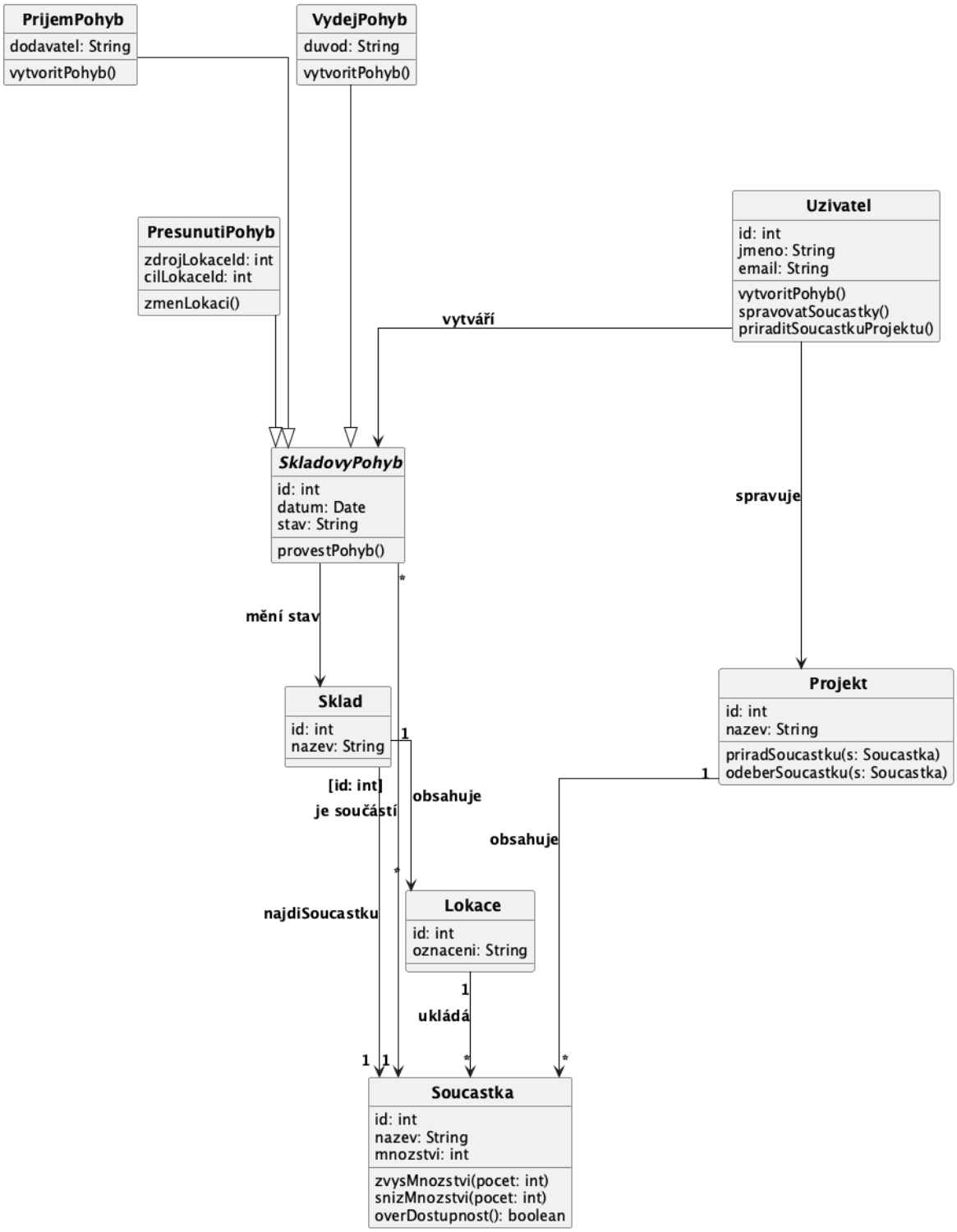
**`Projekt`** Projekt, ke kterému jsou součástky přiřazovány; identifikován pomocí `id` a `nazev`.

**`SkladovyPohyb`** Záznam o pohybu součástky (příjem, výdej, přesun); obsahuje `id`, `typ` pohybu, `datum` provedení a aktuální `stav`.

## 1.9 Diagram tříd

Diagram zachycuje strukturu systému WMS a vztahy mezi jeho hlavními třídami. Součástky jsou přiřazovány k projektům a fyzicky umístěny v lokacích skladu. Třída `Uzivatel` reprezentuje uživatelskou roli v systému – může vytvářet skladové pohyby, spravovat součástky, přiřazovat součástky k projektům a zobrazovat stav skladu.

Diagram využívá **generalizaci**: abstraktní třída `SkladovyPohyb` je základem pro tři konkrétní podtřídy `PrijemPohyb` (naskladnění), `VydejPohyb` (vyskladnění) a `PresunutiPohyb` (přesun mezi lokacemi), které dědí společné atributy a přidávají vlastní specifické atributy. Vazba mezi `Sklad` a `Soucastka` je **kvalifikovaná** – sklad přistupuje ke konkrétní součástce prostřednictvím kvalifikátoru `id: int`.




Obrázek 1.7: Class diagram WMS

## 1.10 Stavový diagram třídy `SkladovyPohyb`

Třída `SkladovyPohyb` reprezentuje obecný skladový pohyb – tedy jakoukoli změnu stavu zásob součástek v systému. Atribut `typ` rozlišuje tři druhy pohybu: *příjem* (naskladnění nových součástek), *výdej* (odebrání součástek ze skladu) a *přesun* (přemístění součástek mezi lokacemi či sklady).

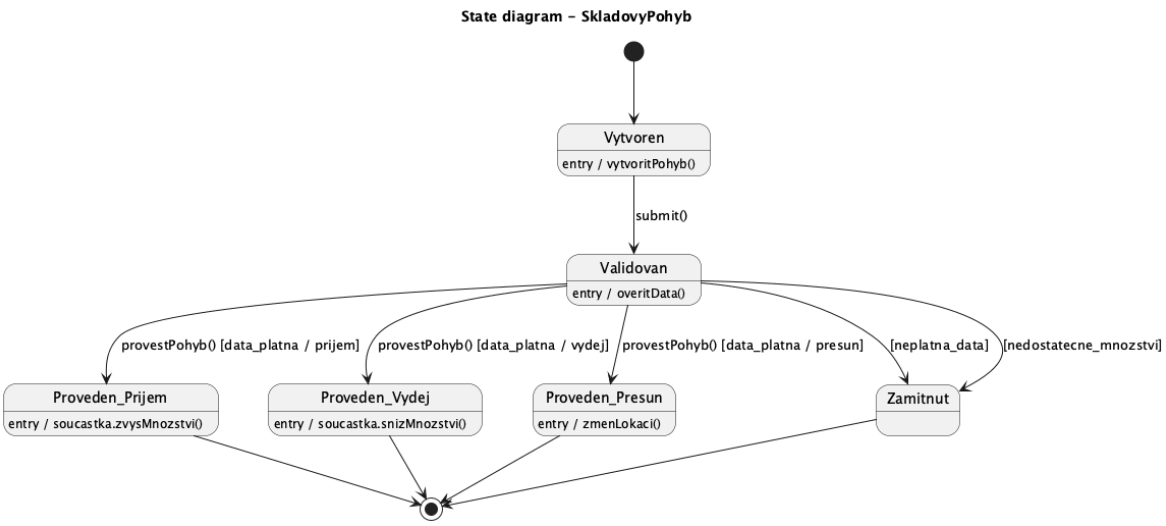
Životní cyklus pohybu je řízen třemi metodami:

**`vytvoritPohyb()`** Inicializuje nový pohyb – přiřadí ID, nastaví datum vzniku, typ pohybu a vazbu na součástku.

**`overitData()`** Ověří platnost zadaných dat. Pohyb je zamítnut pokud:

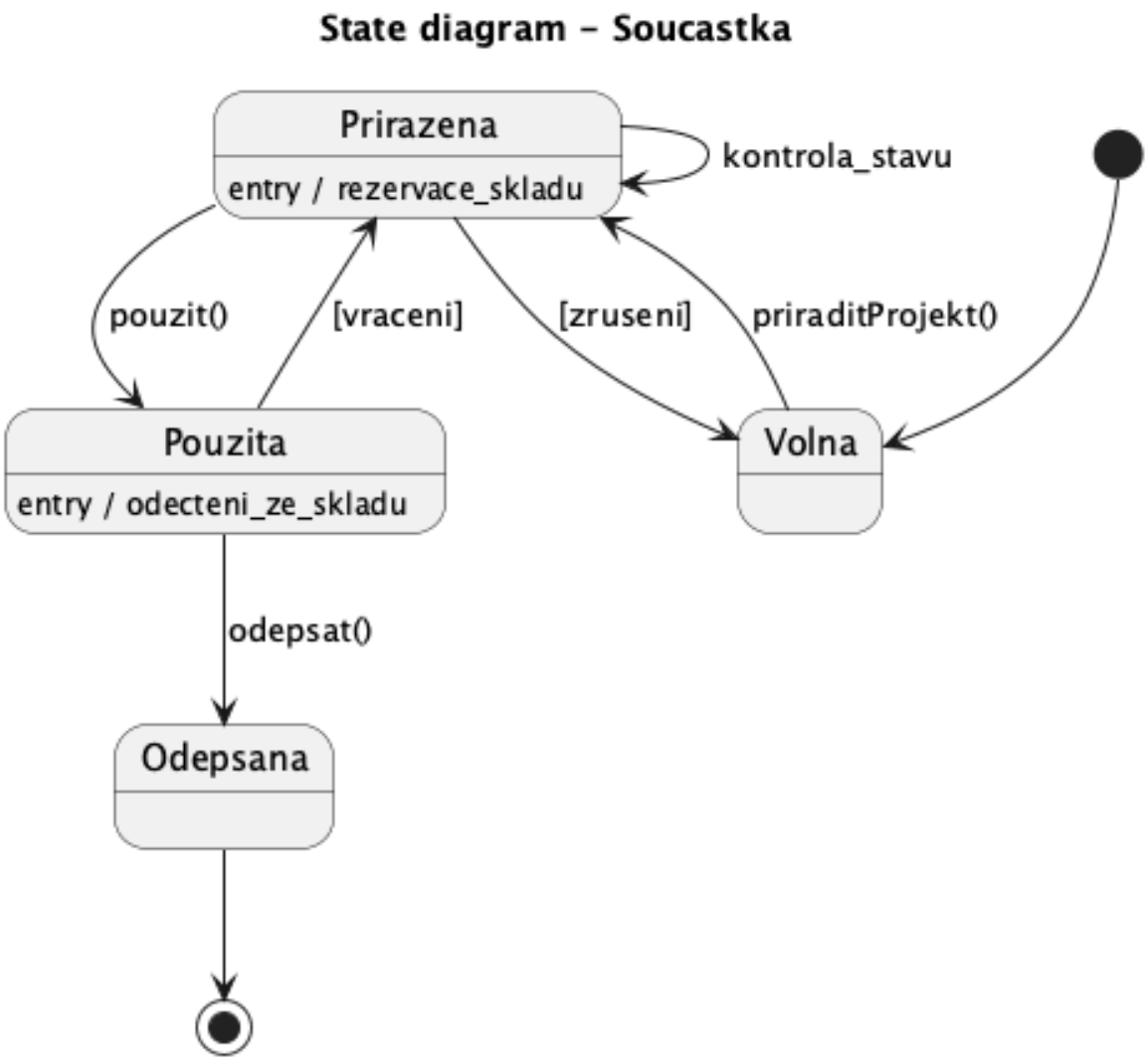
- součástka neexistuje, chybí povinné údaje nebo množství je 0 či záporné (`[neplatna_data]`),
- požadované množství výdeje přesahuje aktuální stav na skladě (`[nedostatecne_mnozstv`
- u přesunu: zdrojová lokace neobsahuje danou součástku nebo cílová lokace neexistuje.

**`provestPohyb()`** Fyzicky provede pohyb: u příjmu zavolá `soucastka.zvysMnozstvi()`, u výdeje `soucastka.snizMnozstvi()` a u přesunu `zmenLokaci()`.



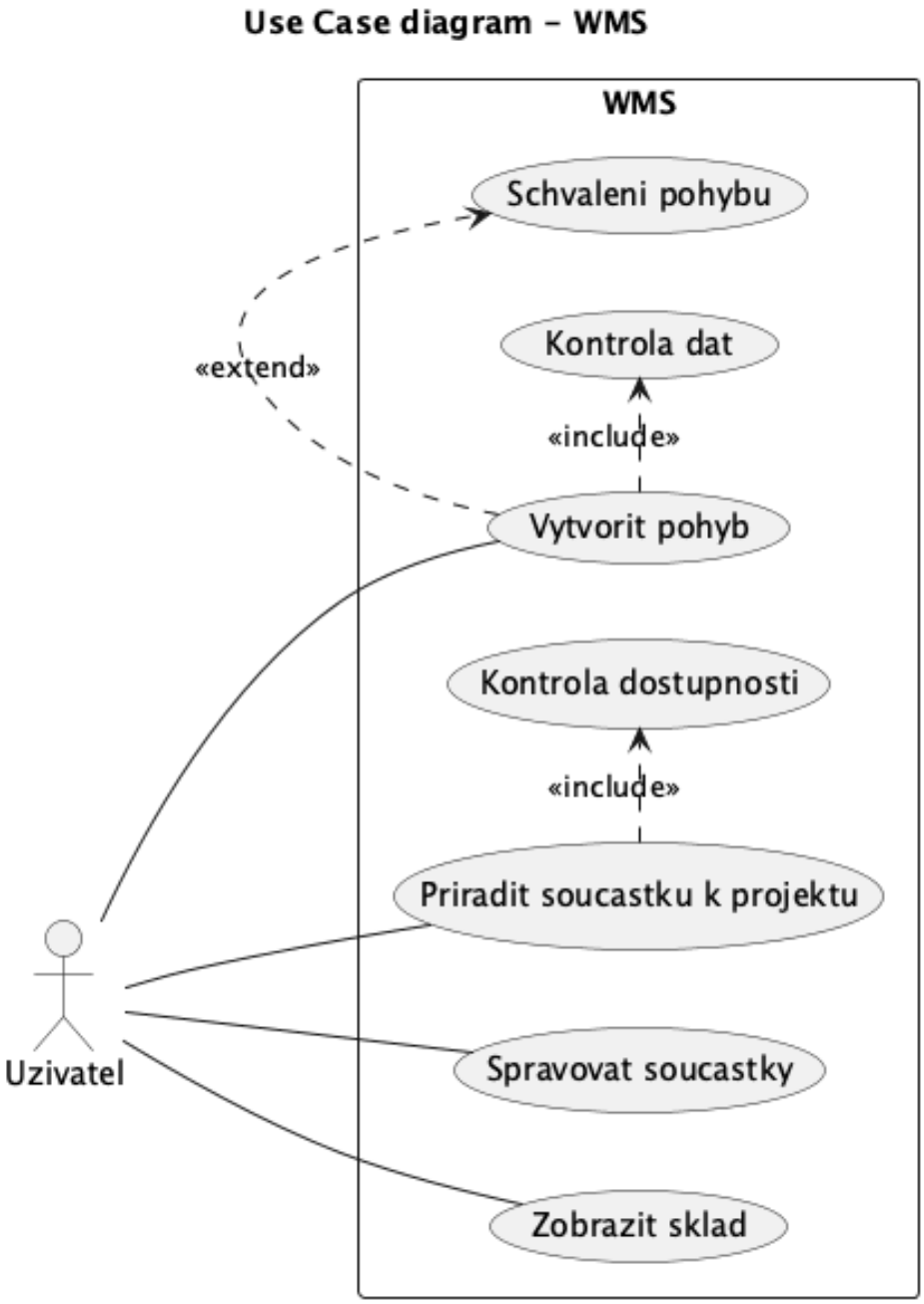

Obrázek 1.8: Stavový diagram třídy SkladovyPohyb

## 1.11 Stavový diagram `Soucastka` (v projektu)



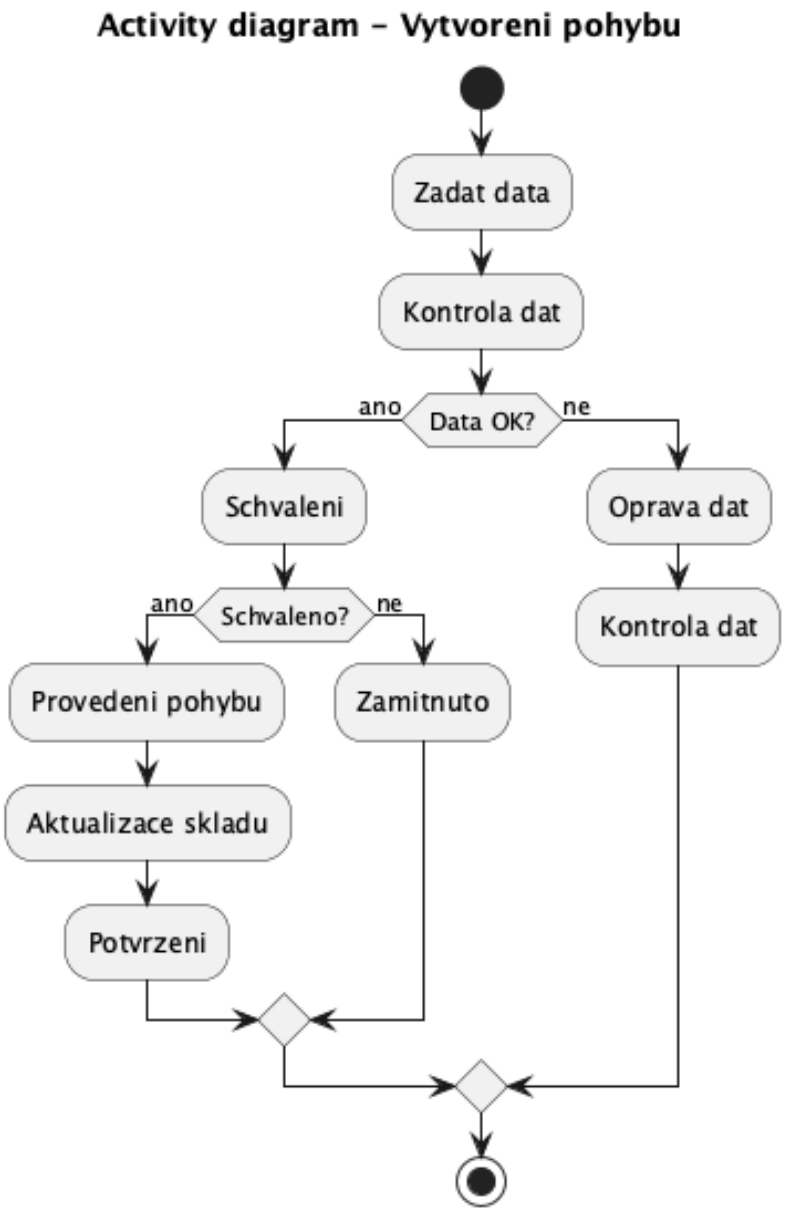

Obrázek 1.9: Stavový diagram třídy Soucastka (v projektu)

## 1.12 Use Case Diagram



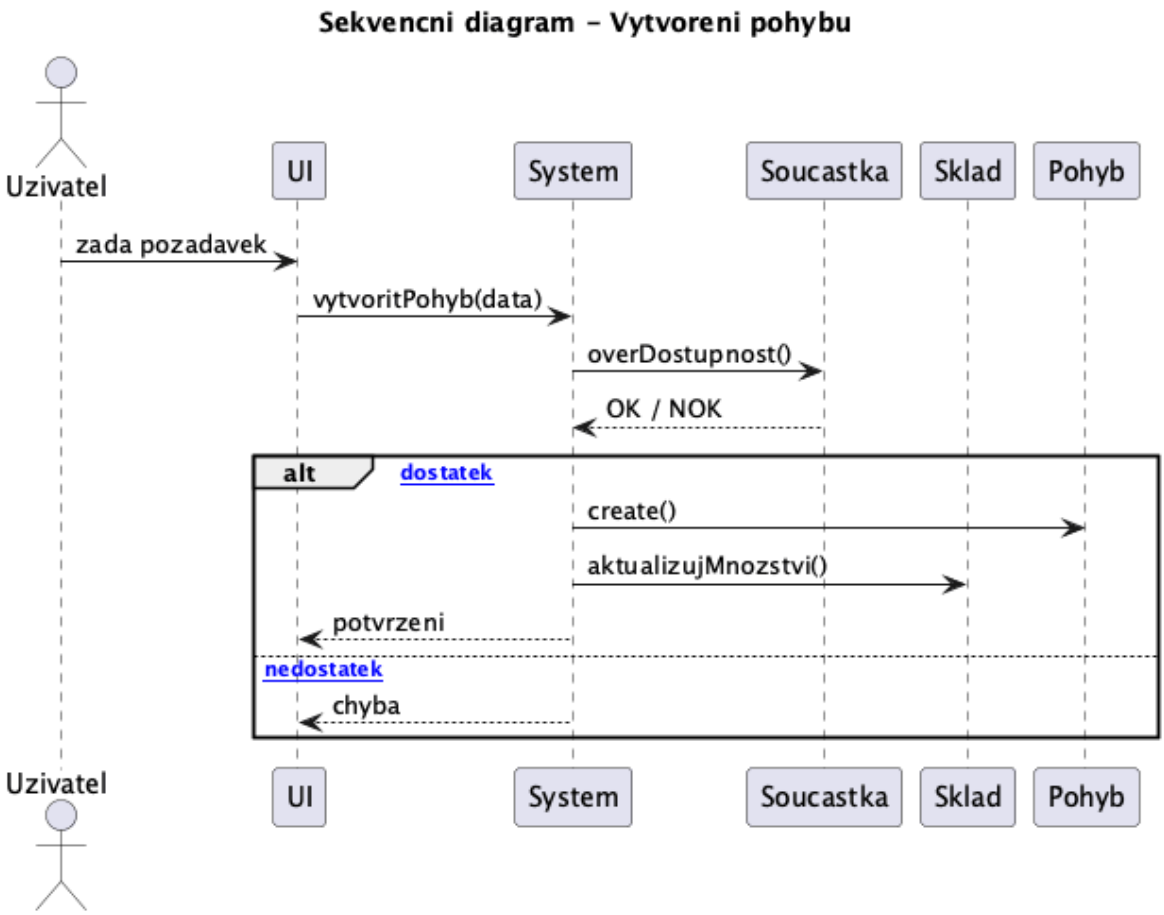

Obrázek 1.10: Use Case Diagram – WMS

## 1.13 Activity Diagram – Vytvoření pohybu




Obrázek 1.11: Activity diagram – Vytvoření pohybu

## 1.14 Sekvenční diagram – Vytvoření pohybu




Obrázek 1.12: Sekvenční diagram – Vytvoření pohybu

## 1.15 Scénáře use case

### Scénář: Vytvoření pohybu

1. Uživatel zvolí *Vytvořit pohyb*.
2. Zadá součástku a množství.
3. Systém provede kontrolu dat.
4. Pokud `[data_ok]` → pohyb je schválen.
5. Systém provede pohyb a aktualizuje stav skladu.
6. Zobrazí se potvrzení výsledku.

### Scénář: Přiřazení součástky k projektu

1. Uživatel vybere projekt.
2. Zvolí součástku a zadá množství.
3. Systém ověří dostupnost voláním `overDostupnost()`.
4. Pokud `[dostatek]` → součástka je přiřazena.
5. Systém aktualizuje množství na skladě.